

VeriHire – Internship & Placement Opportunity Credibility Platform

Project Proposal for UCS503P

Submitted to: Dr. Jeelani

Thapar Institute of Engineering and Technology

Geet Rawal, CSED*

Nidhish, CSED[†]

Parvesh Lamba, CSED[‡]

August 25, 2026

1 Higher-order goal

This project aims to improve the safety and reliability of internship and placement discovery for students by reducing the uncertainty involved in evaluating online opportunities. Students frequently encounter internship and job opportunities through social media, messaging groups, professional networking platforms, emails, and independent websites, where credibility can be difficult to verify.

The immediate engineering objective is to develop a measurable and deployable machine-learning-based software solution that can identify patterns associated with credible and potentially suspicious internship and placement opportunities. The system will provide students with an ML-based assessment that can assist them in deciding whether additional verification is required before applying.

2 Time-to-value

- We will first implement the core workflow of opportunity submission, preprocessing, ML inference, and result visualization.
- We will develop the system through short iterations, allowing both the software workflow and machine learning pipeline to be tested continuously.
- Classical ML models will be implemented as baselines, followed by the proposed semantic-embedding and structured-feature MLP architecture.
- Model performance will be measured using precision, recall, F1-score, confusion matrices, and prediction latency.
- Success is defined by achieving a functional end-to-end ML workflow in the first iteration and improving the model and application through subsequent testing and feedback.

*Roll No: 1024160012, Email: grawl_be24@thapar.edu

[†]Roll No: 1024160013, Email: nnidhish_be24@thapar.edu

[‡]Roll No: 1024160017, Email: plamba_be24@thapar.edu

3 Problem Statement

Students increasingly discover internship and placement opportunities through multiple online channels. However, they often lack a reliable and systematic way to evaluate the credibility of these opportunities before applying.

Common pain points include:

- **Information fragmentation:** opportunity details are often distributed across job posts, company websites, messages, emails, and recruiter profiles.
- **Suspicious requirements:** some opportunities may request registration fees, deposits, paid training, or other forms of payment before selection.
- **Misleading claims:** unrealistic salary promises, guaranteed placements, urgency-based language, or exaggerated benefits can make suspicious opportunities difficult to identify.
- **Unstructured information:** internship descriptions contain textual, numerical, and categorical information that is difficult to evaluate manually.
- **Limited reporting:** students generally lack a centralized platform through which suspicious opportunities can be reported and information can be shared with others.
- **Manual verification:** students often have to independently investigate companies, recruiters, and job descriptions before deciding whether to apply.

Why this matters now (contextual relevance): Students frequently make internship and placement decisions under time constraints and with incomplete information. A machine-learning-based system can learn patterns from labelled historical data and provide an initial credibility assessment, reducing the effort required for manual evaluation.

4 Proposed Solution

4.1 Overview

Build a **web-based** “Opportunity-to-Credibility” platform with the following components:

- **Opportunity submission portal:** students submit internship or placement information such as company, role, description, salary, location, recruiter information, and application details.
- **Data preprocessing:** submitted information is cleaned, normalized, and transformed into textual, numerical, and categorical features suitable for machine learning.
- **ML prediction engine:** the proposed hybrid model combines semantic text embeddings with structured opportunity features and uses a task-specific Multi-Layer Perceptron (MLP) to predict the opportunity’s credibility/risk category.
- **Prediction interface:** the system displays the prediction, confidence/probability information, and supporting indicators to the student without presenting the result as absolute proof of fraud.
- **Community reporting:** students can report suspicious opportunities and provide additional information or evidence.
- **Admin/moderator dashboard:** authorized users can review opportunities, reports, verification decisions, and model predictions.
- **Analytics:** administrators can view statistics related to opportunities, predictions, reports, model performance, and system usage.

4.2 Core workflow

1. A student submits an internship or placement opportunity and receives an opportunity ID.
2. The system validates and normalizes the submitted information.
3. The opportunity text is prepared for semantic embedding while numerical and categorical attributes are encoded as structured features.
4. A pretrained **all-MiniLM-L6-v2** Sentence Transformer generates a semantic representation of the opportunity text.
5. The semantic embedding and encoded structured features are fused and passed to the task-specific MLP classifier.
6. The model generates a predicted credibility/risk category and associated probability/confidence information.
7. Supporting indicators are generated from relevant input features and the prediction is stored in the system.
8. The result is displayed to the student. Students can report suspicious opportunities, while moderators can review reports and verification status.

4.3 Operational constraints for an educational setting

- Keep the solution usable on common student devices through responsive web design.
- Use a suitable labelled dataset for supervised machine learning and maintain a clear separation between training, validation, and testing data.
- Use a pretrained sentence encoder as a feature extractor rather than training a large language model from scratch.
- Keep the task-specific MLP and inference pipeline lightweight enough to provide predictions within an acceptable response time.
- Avoid claiming absolute fraud detection; the system will provide an ML-based assessment to support student decision-making.
- Design the system for deployment using common web hosting, managed databases, and a lightweight Python ML inference service.

5 Solution Approach

This is an engineering course project with a machine learning component. The focus will be on developing a complete, deployable ML-enabled software system rather than solely optimizing a machine learning algorithm.

The overall pipeline will be:

Data Collection → Data Cleaning → Feature Engineering → Model Training → Evaluation → Deployment

5.1 Machine learning pipeline

The problem will be formulated as a **supervised classification problem**. The dataset will contain internship/job opportunity examples with labels representing their credibility/risk category.

Potential features include:

- Internship/job description
- Salary or stipend information
- Payment-related terms and requirements
- Experience requirements
- Company information
- Recruiter/contact information
- Application method
- Location and employment type
- Relevant textual patterns

The project will use a two-stage modeling strategy. First, lightweight classical models will be used as **baselines**. TF-IDF features may be combined with Logistic Regression, Support Vector Machine, Random Forest, or Gradient Boosting to establish reference performance.

The proposed final architecture will use a pretrained **Sentence Transformer, all-MiniLM-L6-v2**, as a semantic feature extractor. Its text embedding will be combined with encoded numerical and categorical opportunity features. The resulting fused feature vector will be passed to a task-specific **Multi-Layer Perceptron (MLP)**.

The proposed MLP architecture is:

Fused Features $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ Output

The output layer will represent the selected credibility/risk classes. The exact hidden-layer configuration may be tuned using validation experiments while keeping the architecture feasible for a semester-long project.

Model comparison will use accuracy, precision, recall, F1-score, confusion matrices, and inference latency. The final architecture will be selected based on predictive performance, generalization, interpretability of the results, and deployment feasibility.

5.2 NLP and feature engineering

Since internship and placement opportunities contain substantial textual information, NLP techniques will be used to convert descriptions into machine-readable representations.

The proposed NLP pipeline is:

Opportunity Text $\rightarrow$ Text Cleaning $\rightarrow$ all-MiniLM-L6-v2 $\rightarrow$ Semantic Embedding

Structured information will be processed separately:

Numerical/Categorical Features $\rightarrow$ Encoding/Scaling $\rightarrow$ Structured Feature Vector

The two representations will then be combined:

Semantic Embedding + Structured Features → Feature Fusion → MLP → Prediction

TF-IDF will be retained as a lightweight baseline for comparison rather than as the primary proposed architecture. This provides an empirical comparison between classical text classification and the proposed semantic feature fusion approach.

5.3 Supporting indicators and responsible interpretation

VeriHire will not present an ML prediction as a definitive statement that an opportunity is fraudulent or genuine. Instead, the system will communicate the predicted credibility/risk category together with supporting indicators derived from the submitted opportunity.

Examples of indicators may include payment requirements, unusual application methods, unrealistic compensation patterns, suspicious textual signals, or other features learned and identified during the project. These indicators are intended to help the student understand why additional verification may be appropriate.

5.4 Web architecture

A modular architecture will consist of:

- **Frontend:** responsive React + Vite interface for students, moderators, and administrators.
- **Backend API:** Node.js and Express.js services responsible for authentication, opportunity management, database operations, reporting, and application logic.
- **ML service:** Python-based FastAPI service responsible for preprocessing, Sentence Transformer embedding generation, structured feature encoding, model loading, and MLP inference.
- **Database:** PostgreSQL database storing users, opportunities, predictions, reports, verification decisions, feedback, and related information.
- **Model artifacts:** versioned sentence encoder configuration, trained MLP parameters, feature encoders/scalers, and model metadata.

The ML service will remain independent from the main application backend, allowing the trained model to be updated without major changes to the frontend or core application.

5.5 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build the frontend and backend and run tests on every change.
- Test ML preprocessing, feature encoding, and prediction functions.
- Produce repeatable deployment artifacts for staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

We define success using measurable metrics associated with both the machine learning model and the core application workflow.

6.1 Primary evaluation metric

ML Classification F1-score: the primary evaluation metric will be the F1-score obtained on a held-out test dataset.

- **Target:** achieve an initial F1-score of approximately 0.80 or above, subject to dataset quality and class distribution.
- **Attribution:** calculated by comparing model predictions against the known labels in the held-out test dataset.

6.2 Secondary evaluation metrics

- **Precision:** proportion of predicted suspicious opportunities that are actually suspicious according to the test labels.
- **Recall:** proportion of suspicious opportunities correctly identified by the model.
- **Inference latency:** time required by the ML service to preprocess an opportunity and return its prediction.
- **Model comparison:** performance difference between the classical baseline models and the proposed embedding + structured features + MLP architecture.
- **User clarity:** student-reported understanding of the prediction using a simple 1–5 feedback question.
- **Reliability:** availability of login, submission, and prediction functionality during the pilot.

6.3 Pilot validation plan

- Prepare a labelled dataset containing credible and potentially suspicious internship/job opportunities.
- Divide the dataset into training, validation, and test sets and compare the classical baselines against the proposed MLP architecture.
- Integrate the selected model into the web application and conduct testing with approximately 10–20 student users.
- Collect feedback regarding usability, prediction clarity, and system performance and use the results for subsequent improvements.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in the number of users and opportunities by:
 - decoupling frontend, backend, ML service, and database,
 - enabling pagination and indexing for common queries,
 - using stateless API design,
 - separating model inference from the main application,
 - maintaining versioned ML models.

2. **Operationally deployable (practically):** The system should run on common cloud infrastructure:

- managed PostgreSQL database,
- platform-hosted frontend and backend,
- Python/FastAPI ML inference service,
- containerized or platform-hosted deployment,
- CI/CD pipeline for staging and production.

8 Engine availability heuristic

A mature set of tools and frameworks is available to implement VeriHire as a machine-learning-enabled web project:

- **React + Vite:** responsive frontend development.
- **Node.js + Express.js:** backend REST APIs and application logic.
- **PostgreSQL:** structured relational database.
- **Python, Pandas and NumPy:** data processing and preparation.
- **Scikit-learn:** baseline model training, preprocessing, evaluation, and comparison.
- **Sentence Transformers:** pretrained semantic text embeddings using `all-MiniLM-L6-v2`.
- **PyTorch:** implementation and training of the task-specific MLP classifier.
- **FastAPI:** serving the trained ML pipeline through an API.
- **Git and GitHub:** version control and collaborative development.
- **Testing and CI tools:** automated unit/integration testing and deployment pipelines.

We will use these mature components to focus on data quality, feature engineering, model evaluation, software architecture, correctness, and measurement rather than developing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Student registration and login.
- Internship/job opportunity submission and opportunity ID generation.
- Opportunity browsing and basic search.
- Dataset preparation, cleaning, and preprocessing pipeline.
- TF-IDF/classical ML baseline for initial performance measurement.
- Sentence Transformer embedding pipeline using `all-MiniLM-L6-v2`.
- Structured feature encoding and feature-fusion pipeline.

- Task-specific MLP classifier and evaluation pipeline.
- Prediction API using Python/FastAPI.
- Web interface for displaying the ML prediction and supporting indicators.
- PostgreSQL database for users, opportunities, predictions, and reports.
- CI: build, linting, and backend/ML unit tests.
- CD to staging with a single command or pipeline job.

9.2 Subsequent deliverables

- Hyperparameter tuning and comparison of baseline and MLP models.
- Improved NLP and structured feature engineering.
- Prediction interpretation and supporting indicators.
- Community reporting and moderator review.
- Company verification and evidence management.
- Advanced search and filtering.
- Notification layer using in-app notifications, with email as an optional later enhancement.
- Admin dashboard for model, opportunity, and moderation analytics.
- Model version management and improved scale readiness.

10 Risks and mitigations

- **Dataset quality:** poor or biased training data can reduce model performance. **Mitigation:** perform data cleaning, exploratory analysis, duplicate removal, and appropriate dataset splitting.
- **Class imbalance:** one class may contain significantly more examples than another. **Mitigation:** analyze class distribution and use class weighting or suitable resampling techniques where required.
- **Overfitting:** the model may perform well on training data but poorly on unseen opportunities. **Mitigation:** use validation and test datasets, cross-validation where appropriate, dropout or other regularization techniques, and early stopping where applicable.
- **False predictions:** the model may incorrectly classify legitimate or suspicious opportunities. **Mitigation:** evaluate precision, recall, F1-score, and confusion matrices and clearly present predictions as decision support.
- **Model drift:** suspicious opportunity patterns may change over time. **Mitigation:** design the system so that new labelled data can be used for future model retraining.
- **Data privacy:** user submissions may contain personal or company information. **Mitigation:** minimize collected data and apply authentication and access controls.
- **Operational issues in pilot:** ML service or deployment failures may affect demonstrations. **Mitigation:** deploy the inference service early, test the prediction API independently, and maintain a stable model version for demonstration.

11 Summary

This project proposes **VeriHire**, a deployable machine-learning-based web platform designed to help students evaluate the credibility of internship and placement opportunities.

The final proposed ML workflow is:

Opportunity Text $\rightarrow$ all-MiniLM-L6-v2 Embedding

Structured Features $\rightarrow$ Encoding/Scaling

Semantic Embedding+Structured Features $\rightarrow$ Feature Fusion $\rightarrow$ MLP $\rightarrow$ Credibility/Risk Prediction

The project will use classical TF-IDF-based models as baselines and compare them with the proposed semantic-embedding and structured-feature MLP architecture. The final model will be selected using precision, recall, F1-score, confusion matrices, inference latency, and deployment feasibility.

It meets the course criteria by emphasizing rapid iterations, modular software architecture, machine learning integration, CI/CD, testing, scalability, and measurable evaluation criteria.

The proposed technology stack consists of **React, Vite, Node.js, Express.js, PostgreSQL, Python, Pandas, NumPy, Scikit-learn, Sentence Transformers, PyTorch, FastAPI, and Git/GitHub**. The project is feasible for a team of 3–4 students within one semester, with advanced explainability, analytics, moderation, notifications, and model improvements introduced incrementally.

The ultimate objective is to help students move beyond simply asking “Is this internship genuine?” and instead receive a data-driven assessment that helps them make a more informed decision before applying.